
CSE 234 Project 2 Report

Ewan Shen

Department of Computer Science
University of California, San Diego
ewshen@ucsd.edu

Shinjini Halder

Department of Computer Science
University of California, San Diego
shhalder@ucsd.edu

1 Training Data Methodology

We approached the project as a schema-linking task where natural language questions pair to a database schema and *schema.link* JSON object. Our pipeline outputs table names that are paired to referenced column names. Furthermore, tables with no matched columns are treated as an empty list. Regarding how we composed our training data, we used the *train.json* JSON file with 301 examples in 17 schemas in the project packet as the training set for supervised learning. Each of the 301 examples contains an ID, natural language query, the corresponding golden SQL query, and the schema links.

We first converted the schema to table and column matches and use the *pk_fk* serialization format. *pk_fk* includes primary key columns and foreground key links between tables in addition to the tables and columns. We used the method because column-level schema linking extends beyond only finding columns that are directly mentioned in the question. Many golden schema links include key columns that connect tables in SQL. Our use of *pk_fk* serialization lists primary keys and foreign keys, which allows our model to reason tables that are related.

To filter for relevant schemas, we used lexical filtering, which scored tables according to token overlap with the specific question. To train the adapter, we chose a top k value of 24 after experimentation. Selecting only the most relevant tables for a given question reduces noisy tables while still providing enough relevant information. We also included alias-boosted retrieval for schema filtering. Specifically, some keywords and abbreviations in the tables were not being covered by token overlap with the question. Inspired by sparse search methods for RAG pipelines, we added alias boosting which corresponded phrases such as "crash avoidance" to relevant tables with abbreviations that would otherwise not be picked up through token overlap.

For building the prompt, each training example included task instructions with constraints such as the requirement to only output valid JSON, the serialized schema, and the natural language query. Each prompt was matched with the golden schema links that were serialized as JSON.

Regarding imbalances in the training data, we noticed through our experiments that certain schemas were failing, especially those that were underrepresented, such as SBOD. We balanced the dataset by $3x$ oversampling from SBOD examples and the related multi-table examples. Oversampling from schemas low in number allows training to be more balanced across the schemas to allow the model to reason across the knowledge domains in a generalized and balanced manner. We chose to focus efforts on implementing and testing oversampling rather generating synthetic training examples to avoid potential noisy generations and incorrect labels. Data quality matters more than data quantity, and as a result, we focused on *pk_fk* serialization, relevant table filtering, and dataset balancing through oversampling for high quality training data.

Regarding diagnostic metrics for question error analysis, we wrote a custom script that consumed the prediction JSON files and the per-question CSV outputs after running the evaluation script. Our error analysis script compared each prediction against the golden schema links and reported the incorrect number of tables and columns for each question. Running our own error analysis scripts revealed recurring failure modes such as missed coded table names.

2 Final Pipeline Architecture

For our final pipeline architecture, we used *Qwen2.5 – 1.5B – Instruct* as our base model. The model was chosen due to it being instruction-tuned and below the $2B$ parameter budget. In our pipeline, the model is loaded from Hugging Face during inference.

We used LoRA to train an adapter rather than full fine-tuning to lower computational costs and training time as not all the parameters need to be updated during training. After experimenting with multiple LoRA ranks, alphas, and dropout values, we used $r = 32$, $lora_alpha = 64$, and $lora_dropout = 0.05$ to train the adapter. Additionally, the adapter was trained on schemas serialized with *pk_fk* and the top 24 tables for each question as described in section 1. The final adapter is stored locally and loaded at inference with PEFT.

Regarding our prompt template, we first included the instruction to the model that it was tasked with a schema-linking model role. We followed the role instruction with the overall task description of only outputting JSON that maps tables to columns. We also included constraints for specification, including the requirement to only use table and column names present in the schema, to represent tables with no matched columns as empty lists, to not include extra keys, and to prefer the most specific tables with directly relevant columns.

In our pipeline, our inference script, *main.py*, first loads the schema for the given question. Tables are then ranked by token overlap with the given question. To consider abbreviations, we use alias boosting to increase table scores with relevant domain phrases that we determined through debugging. We then choose the top 25 tables to be serialized into the prompt. The k value was picked after experimentation of multiple *infer_k* values and we converged on $k = 25$ for inference.

Regarding output decoding, we set the maximum tokens to 192 after experimenting and determining that increasing the maximum number of tokens did not change accuracy. We also used a temperature of 0.0 to avoid stochastic sampling for deterministic experimentation. The model output text is decoded after the input prompt tokens and the decoder extracts the first JSON object from the text.

Regarding post-processing, the parser first parses for strict JSON. If the strict parsing fails, JSON repair occurs by removing trailing commas and brackets, deleting Markdown code fences, and handling Python-style dictionary syntax. After a JSON object is recovered, the prediction is validated against the database schema where table and column names are matched and converted back to schema-original casing. If no valid JSON object can be recovered, our pipeline falls back to lexical schema matching so that *main.py* still produces a valid *schema_links* object. For identifier validation, as mentioned, the table names are matched and converted back to the original casing used in the schema file after parsing. Column names are matched to within the specific table. All matching is done in a case sensitive manner. Identifiers that does not exist in the schema are dropped before the final prediction is written to prevent hallucinated identifiers from appearing in the output. Finally, the output is written to a JSON file.

3 Experimentation Methodology

Table 1: Schema Linking SFT and Inference Configuration Experiments

Run	Base Model	Train Alias	Infer Alias	Infer k	Max Tokens	Schema Format	Epochs	LoRA r	LR	Balance Strategy	Score
1	Qwen2.5-1.5B	False	False	25	192	pk_fk	3	32	1e-4	SBOD x3 + MT x3	0.5030
2	Qwen2.5-1.5B	False	True	25	192	pk_fk	3	32	1e-4	SBOD x3 + MT x3	0.5208
3	Qwen2.5-1.5B	False	True	20	192	pk_fk	3	32	1e-4	SBOD x3 + MT x3	0.5138
4	Qwen2.5-1.5B	False	True	32	192	pk_fk	3	32	1e-4	SBOD x3 + MT x3	0.5044
5	Qwen2.5-1.5B	False	True	25	256	pk_fk	3	32	1e-4	SBOD x3 + MT x3	0.5208
6	Qwen2.5-1.5B	True	True	25	192	pk_fk	3	32	1e-4	SBOD x3 + MT x3	0.5219
7	Qwen2.5-1.5B	True	True	25	192	pk_fk	4	32	1e-4	SBOD x3 + MT x3	0.4333
8	Qwen2.5-1.5B	True	True	25	192	basic	3	32	1e-4	SBOD x3 + MT x3	0.4970
9	Qwen2.5-1.5B	True	True	25	192	pk_fk	3	16	1e-4	SBOD x3 + MT x3	0.4610
10	Qwen2.5-1.5B	True	True	25	192	pk_fk	3	32	5e-5	SBOD x3 + MT x3	0.4188

For the baseline, we considered lexical schema retrieval, schema serialization, and constrained JSON prediction generation. To improve schema-linking performance, we conducted a systematic series of training and inference experiments using a LoRA fine-tuned version of Qwen2.5-1.5B-Instruct.

One of the primary factors investigated was schema retrieval size. Schema filtering selects the most relevant tables before prompting the model. Increasing the retrieval window initially improved performance because the model gained access to a larger portion of the relevant schema and was less likely to miss necessary tables. However, after a certain point, performance began to decline because

additional retrieved tables acted as distractors, introducing retrieval noise and reducing precision. Experiments with *schema_top_k* values of 20, 25, and 32 showed that a moderate retrieval size provided the best balance between schema coverage and noise.

We also evaluated different schema serialization strategies. Adding primary-key and foreign-key information improved performance because it provided explicit relational structure, helping the model reason about joins and relationships between tables. In contrast, using only basic schema descriptions removed these relational cues and reduced linking accuracy.

Another factor was alias-aware matching improved performance because many schemas contain abbreviated or domain-specific identifiers that do not directly match the wording used in natural-language questions. By expanding lexical matches between user phrases and schema identifiers, alias boosting increased both table and column retrieval accuracy without introducing invalid schema elements.

Several LoRA training configurations were explored. Increasing the adapter rank from 16 to 32 improved performance because the model had greater capacity to learn schema-linking patterns and database-specific naming conventions. Training duration was also important. The 3-epoch alias-trained model achieved the best validation score, while extending training to 4 epochs reduced the score substantially to 0.4333. This suggests that the small training set was vulnerable to overfitting and that additional training did not improve generalization.

To better understand the sensitivity of the model to training hyperparameters, we varied the number of training epochs, LoRA rank, and learning rate. Reducing the LoRA rank from 32 to 16 reduced performance, indicating that the higher-rank adapter provided useful capacity for learning database-specific schema-linking patterns. Lowering the learning rate from 1e-4 to 5e-5 also hurt performance, suggesting that the lower rate did not adapt the LoRA parameters sufficiently within the fixed 3-epoch budget.

Finally, we investigated data-centric balancing choices through SBOD and multi-table oversampling. The final configuration used 3x oversampling for sparse SBOD modules and 3x oversampling for examples involving at least two linked tables. We did not use synthetic augmentation or paraphrase generation; instead, we focused on balancing the released gold examples and improving schema serialization and retrieval.

3.1 Evaluation Procedure

We performed the multi-configuration experimentation through reproducible command-line runs rather than through RapidFire AI. All experiments were evaluated on the provided validation split using the official schema-linking evaluation pipeline. For each question, the model predicted the set of relevant tables and columns required to answer the query. Performance was measured using three metrics: table score, column score, and the overall leaderboard score.

The table score evaluates how accurately the model identifies the relevant database tables, while the column score measures the correctness of the predicted columns. The leaderboard score combines these aspects into a single metric and was used as the primary criterion for comparing experimental configurations.

To ensure fair comparison, each experiment was evaluated using the same validation set and scoring procedure. During experimentation, only one major component was modified at a time, such as retrieval size, alias handling, training configuration, data augmentation strategy, or post-processing repair. This allowed us to isolate the contribution of each change and understand its impact on overall schema-linking performance.

The best-performing configuration from each stage was carried forward to subsequent experiments, resulting in a progressive refinement of the final pipeline.

Additionally, all prediction files were validated through a schema canonicalization pipeline that ensured generated identifiers corresponded to valid schema elements. Across all experiments, the number of invalid table and column identifiers remained zero, eliminating schema hallucination as a major source of error.

For each configuration, we saved the training command when applicable, the prediction JSON file, and the official validation evaluation output in the repository logs. This gave us a systematic

comparison across more than eight distinct configurations while keeping each run reproducible from the submitted scripts.

4 Results and Analysis

Run	Change Tested	Key Setting	Table Score	Column Score	Leaderboard
Exp0 1	Old best adapter, no alias boost	k=25 , pk_fk	0.5999	0.4061	0.5030
Exp0 2	Add inference alias boost	k=25 , pk_fk , alias boost	0.6270	0.4147	0.5208
Exp0 3	Smaller retrieval window	k=20 , alias boost	0.6087	0.4189	0.5138
Exp0 4	Larger retrieval window	k=32 , alias boost	0.6011	0.4078	0.5044
Exp0 5	Longer generation	max_new_tokens=256	0.6270	0.4147	0.5208
Exp0 6	Alias-aware trained adapter	train + infer alias, k=25	0.6320	0.4118	0.5219
Exp0 7	More epochs	4 epochs	0.5442	0.3224	0.4333
Exp0 8	Basic schema format	no pk_fk	0.6117	0.3824	0.4970
Exp0 9	Lower LoRA rank	r=16	0.5622	0.3599	0.4610
Exp10	Lower learning rate	lr=5e-5	0.5149	0.3227	0.4188

Figure 1: Summary of schema-linking experiments and their validation performance.

After running a series of experiments, the best performing configuration achieved a validation score of 0.5219. This was reached by combining several carefully tuned components, lexical schema retrieval pulling in the top 25 tables, primary/foreign key formatting, alias boosting, a LoRA rank of 32, and a temperature of 0. While the improvement over the previous best score of 0.5208 may seem small, it confirms that alias-aware training consistently added value.

One of the clearest findings was how sensitive the system was to how many tables were retrieved during inference. Retrieving too few tables (20) meant the model occasionally missed tables it actually needed. Retrieving too many (32) introduced noise that confused it. Landing at 25 gave the model enough context without overwhelming it.

Many real-world databases use shorthand or technical column names that don’t match how a person would naturally phrase a question. By boosting matches through aliases, the system became much better at connecting user language to the right schema element, increasing score from 0.5030 to 0.5208 without any additional training.

Many schemas contained abbreviated or domain-specific identifiers that did not directly match user language. Alias boosting improved lexical matching between user queries and schema elements, increasing both table and column identification accuracy without introducing invalid identifiers.

Schema formatting improved result. Including primary and foreign key relationships in the schema prompt gave the model the relational context it needed to reason about joins, increasing the score from 0.4970 up to 0.5219.

On the training side, LoRA rank 32 proved necessary. Dropping to rank 16 caused a significant performance drop, suggesting the task is complex enough that the model genuinely needs the extra capacity to learn database-specific patterns. Similarly, training for more than three epochs led to overfitting, and using a lower learning rate left the model underfit.

Balancing through oversampling was part of the final training setup rather than a separate synthetic augmentation strategy. We oversampled sparse SBOD schemas and multi-table examples to reduce skew in the small released training set. This helped define a stronger training distribution, but the remaining errors show that column-level selection remained substantially harder than table-level selection.

The broader takeaway is that no single factor drove performance. The best results came from getting many things right at once: moderate retrieval, relational schema context, alias matching, and a well-calibrated training setup.

5 AI Tools Statement

Codex and Claude were used for general assistance and question answering. Project specifications were inspected by LLMs to answer general questions. We also used LLMs to help implement and debug code. For example, when we were obtaining low leaderboard scores, we used an error analysis script to expose failure modes and received suggestions from LLMs for potential remediations.